

# KMeansHomework

August 7, 2023

## 1 Bài tập về nhà Kmeans

### 1.1 Mục tiêu

- Tự viết lại code cho giải thuật K-means
- Hiểu sâu hơn giải thuật K-means qua việc tự viết lại code
- Ứng dụng mô hình tự viết vào các bài toán đã ra trên lớp

### 1.2 Dữ liệu

Giống dữ liệu của bài thực hành trên lớp (dữ liệu sinh ngẫu nhiên bằng sklearn và ảnh bird\_small.png)

### 1.3 Yêu cầu

Code K-means tự viết cho kết quả tương đương (không cần giống hệt) với giải thuật của thư viện sklearn khi áp dụng cho dữ liệu sinh ngẫu nhiên và dữ liệu ảnh.

## 2 Các bước làm

### 2.1 Các thư viện sử dụng

```
[ ]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics.pairwise import euclidean_distances
%matplotlib inline
```

### 2.2 Chuẩn bị dữ liệu

- Sinh dữ liệu ngẫu nhiên `n_samples = 100` tương đương 100 điểm
  - `random_state`: biến cố định hàm random - để các điểm sinh ngẫu nhiên giống nhau giữa các máy tính
- Mỗi điểm dữ liệu có 2 chiều

```
[ ]: n_samples = 100
random_state = 170
```

```

center_points = [[1, 1], [-1, -1], [1, -1]] # sinh ngẫu nhiên các điểm xung
↳ quanh vị trí tâm cố định
#center_points = 3 # tâm cụm được chọn ngẫu nhiên

X, y = make_blobs(n_samples=n_samples, random_state=random_state,
↳ centers=center_points, cluster_std=0.6)
print("Số chiều dữ liệu: ", X.shape, y.shape)
print("5 điểm dữ liệu đầu tiên: \n", X[:6])

```

Số chiều dữ liệu: (100, 2) (100,)

5 điểm dữ liệu đầu tiên:

```

[[ 1.26241305  0.94872541]
 [-0.39743873 -1.18567406]
 [ 1.35081331  0.48041993]
 [ 1.21219555  0.98929291]
 [-0.75344338 -1.09784774]
 [ 2.67199591 -0.16659988]]

```

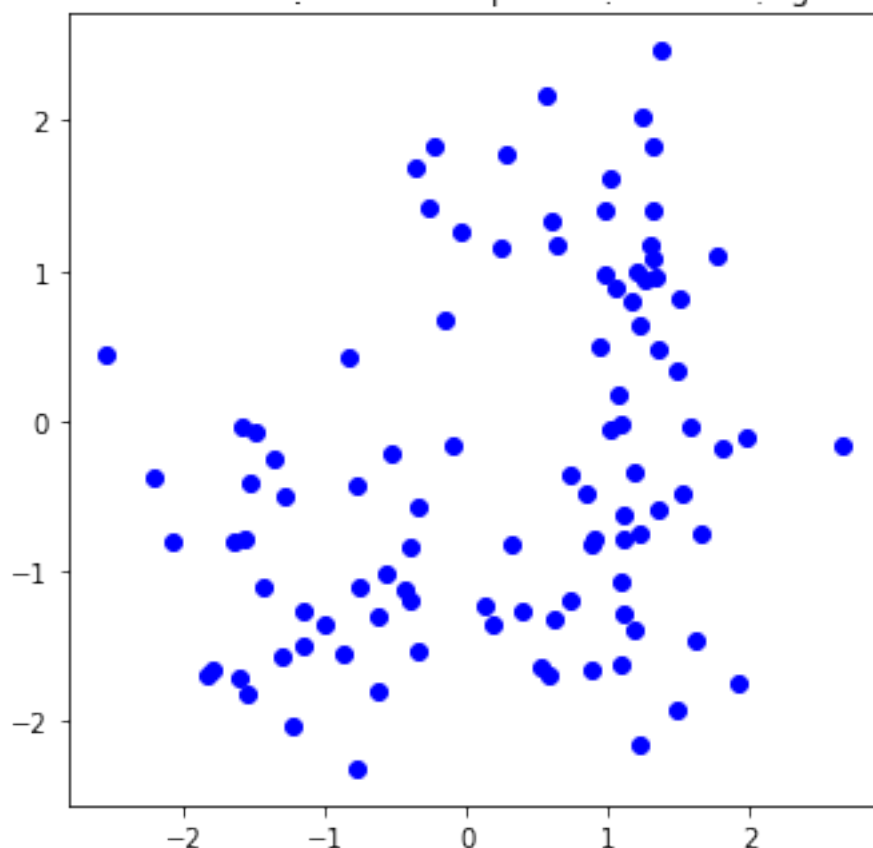
Vẽ các điểm ảnh sử dụng matplotlib

```

[ ]: plt.figure(figsize=(12, 12))
plt.subplot(221)
plt.scatter(X[:, 0], X[:, 1], c='blue') # c là tham số chọn màu sắc, có thể
↳ truyền vào string hoặc số id 1,2,3 ...
plt.title("Các điểm dữ liệu trước khi phân cụm. Số lượng: {}".format(n_samples))
plt.show()

```

Các điểm dữ liệu trước khi phân cụm. Số lượng: 100



## 2.3 Tự xây dựng giải thuật K-means:

Viết code cho giải thuật K-means tại mục này

## 2.4 Kiểm tra giải thuật K-means tự viết cho dữ liệu sinh ngẫu nhiên

- Áp dụng giải thuật K-means tự viết cho tập dữ liệu đã sinh ngẫu nhiên ở trên
- Quan sát kết quả và so sánh với giải thuật của sklearn

## 2.5 Ứng dụng K-means tự viết vào nén ảnh

### 2.5.1 Thư viện sử dụng - hỗ trợ hình ảnh

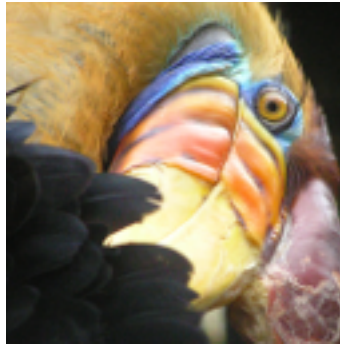
```
[ ]: from skimage import io
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as image
from IPython.core.display import Image, display
```

### 2.5.2 Đọc dữ liệu hình ảnh

- Mỗi điểm ảnh là 1 mẫu quan sát
- Phân cụm tập dữ liệu (tập các điểm ảnh) về k nhãn

```
[ ]: path_img = 'bird_small.png'
display(Image(path_img, width=250, unconfined=True))
img = io.imread(path_img)
data_img = (img / 255.0).reshape(-1, img.shape[2]) # chuyển ma trận 128x128x3 về
↳ mảng 2 chiều
img_shape = img.shape

print("Số chiều của dữ liệu hình ảnh: ", data_img.shape)
print("Tổng số điểm ảnh là: ", data_img.shape[0])
print("Mỗi điểm ảnh có số chiều = ", data_img.shape[1])
```



Số chiều của dữ liệu hình ảnh: (16384, 3)

Tổng số điểm ảnh là: 16384

Mỗi điểm ảnh có số chiều = 3

### 2.5.3 Nén ảnh bằng giải thuật K-means tự viết

- Tạo file nén ảnh bằng giải thuật K-means tự viết
- Hiển thị kết quả của giải thuật tự viết và giải thuật của sklearn để so sánh

Ví dụ

```
[ ]: print('Ảnh nén bằng K-means tự viết')
display(Image('img_128.png', width=250, unconfined=True)) #kết quả tự cài đặt
print('Ảnh nén bằng K-means của thư viện ')
display(Image('img128.png', width=250, unconfined=True)) #kết quả của thư viện
print('Ảnh gốc')
display(Image(path_img, width=250, unconfined=True))
```

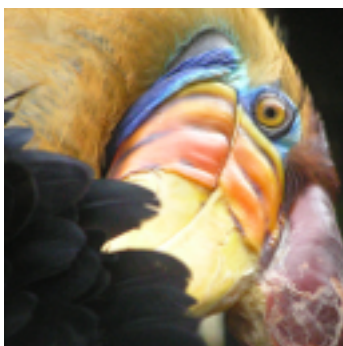
Ảnh nén bằng K-means tự viết



Ảnh nén bằng K-means của thư viện



Ảnh gốc



[ ]: